

Red Hat AI (RHAI) 3.5

New Key Features

Version 1, 2026-08-25

Home

Introduction

Course Title: Red Hat AI (RHAI) 3.5: New Key Features

Description: Welcome to the Red Hat AI (RHAI) 3.5 Enterprise Technical Training Course. This training guide covers the key architectural and functional updates in version 3.5.

This course is designed around the four core product pillars that drive the RHAI 3.5 strategy:

- Fast & Efficient Inference
- Connecting Data to Models
- Agentic AI
- Platform Infrastructure, Scale & Security

Each module represents a product pillar. Within each module, lessons are structured around the official Key Messages, with technical explanations, operational mechanics, and enterprise use-case scenarios.

Duration: >...

Technology Preview Features

Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scop](#).

Developer Preview Features

Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to functionality in advance of possible inclusion in a Red Hat product offering. Customers can use these features to test functionality and provide feedback during the development process. Developer Preview features might not have any documentation, are subject to change or removal at any time, and have received limited testing. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA. For more information about the support scope of Red Hat Developer Preview features, see [Developer Preview Support Scope](#).

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Sarvesh Pandit (Principal Technical Training Developer)
- Carlos Condado (Senior Product Marketing Manager, AI)
- Will McGrath (Senior Principal Product Marketing Manager, AI)
- Martin Isaksson (Principal Architect, AI)
- Younes Ben Brahim (Principal Product Marketing Manager, AI)
- Rutuja Deshmukh (Technical Training Developer)
- Anna Swicegood (Learning Product Manager)



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Fast & Efficient Inference

Production-ready agentic AI requires inference capabilities that go beyond simple text completion. Red Hat AI 3.5 gives you the operational depth, model safety transparency, resource governance, and multimodal foundations to deploy and manage high-priority LLM workloads at scale.

This section focuses on:

- Model Safety and Reliability Evidence Before Deployments
- The Operational Controls Shared Inference Requires
- The Foundation for AI Agents that Work Beyond Text

Model Safety and Reliability Evidence Before Deployments

Enterprise teams in regulated industries need to verify that a model behaves safely and reliably under adversarial conditions before promoting it to production.

Garak Security & Safety Benchmarking Integration [GA]

The Governance Rule

"Verify that a model behaves safely and reliably under adversarial conditions before promoting it to production."

Persona Tension

SRE Lead: "How can we verify that a model behaves safely under adversarial conditions before promoting it to production in a regulated environment"

Platform Engineer: "We integrate Garak benchmark results natively into the Model Catalog under the Safety & Security Insights tab to display a detailed risk profile for every validated model."

Think First: Question

Before examining the mechanics below: How does the Model Catalog display security risk profiles without requiring manual red-teaming for every imported model?

The Core Answer

"The automated validation pipeline executes pre-configured Garak scans via the EvalHub REST API, ingesting structured JSON payloads directly into the Model Catalog UI."

What is it?

Garak (Generative AI Red-teaming & Assessment Kit) is an open-source security scanning framework specifically designed to probe LLMs for vulnerabilities. In RHAI 3.5, Garak benchmark results are natively integrated into the platform's Model Catalog under the Safety & Security

Insights tab, providing a detailed risk profile for every validated model.

How it works?

During the automated model validation pipeline, the model validation team triggers pre-configured security evaluation collections (red teaming, prompt injection, jailbreaking, and data privacy/PII leakage scans) via the EvalHub REST API. These scans run adversarial prompts against the model endpoint. The resulting structured JSON payloads are ingested by the Model Catalog’s BFF (Backend-for-Frontend) and displayed in the UI as an Evaluation benchmarks table. This table includes columns for evaluation name, category, benchmark, description, and numeric result score.

Enterprise Use Case Scenario

A financial services institution is deploying an on-premise, air-gapped LLM to assist with retail banking customer queries. Due to strict compliance requirements under the EU AI Act and NIST AI RMF, they cannot pull models from public repositories without proof of safety. Using the Model Catalog’s Safety & Security Insights tab, platform administrators can immediately verify the model’s resistance to jailbreaking and prompt injection, bypassing weeks of manual red-teaming and legal approvals.

Tool-Calling Validation [GA]

The Developer’s Challenge

"What prevents our API calls from failing silently when models generate malformed arguments or invalid JSON schemas during execution?"

Architectural Dimension	Traditional Validation	RHAI 3.5 Innovation
Tool Execution	Custom client-side parsing code required to handle malformed arguments.	Tool-Calling Validation: Confirmed models display validated vllm serve arguments directly in the catalog.

Think First: Question

Before examining the mechanics below: How do developers ensure model tool invocations conform to expected schemas without writing custom client-side parsing code?

The Core Answer

"By selecting a model labeled with the Tool calling task filter in the RHAI Catalog, the development team starts with a pre-configured template, ensuring that tool invocations conform to the expected schema without custom client-side parsing code."

What is it?

Tool-Calling Validation is a quality verification layer in RHAI 3.5 that guarantees selected models are capable of reliably executing tool calls (such as API invocations and database queries) under standard enterprise conditions.

How it works?

An initial set of high-demand models is validated by the Red Hat Model Validation team specifically for tool-calling capabilities. This validation tests whether a model correctly structures invocations, accurately passes arguments, and correctly handles tool response schemas. Models with confirmed support display a Validated Arguments section on their Model Details page in the Model Catalog. Users can expand the Tool Calling panel to view and copy the exact vllm serve CLI arguments (e.g., `--tool-call-parser`, `--chat-template`, `--enable-auto-tool-choice`) required for correct deployment.

Enterprise Use Case Scenario

A logistics company builds a customer service agent that needs to invoke a proprietary parcel-tracking API. If the model generates malformed arguments or invalid JSON schemas, the API call fails silently. By selecting a model labeled with the Tool calling task filter in the RHAI Catalog, the development team starts with a pre-configured template, ensuring that tool invocations conform to the expected schema without custom client-side parsing code.

The Operational Controls Shared Inference Requires

Enterprise teams run agentic pipelines, batch processing jobs, and latency-sensitive API calls on shared infrastructure. RHAI 3.5 gives you the operational controls to keep competing workloads from disrupting one another.

SLO-Aware Serving [GA]

Architectural Dimension	Traditional Serving	RHAI 3.5 Innovation
Traffic Prioritization	Single tenant monopolization; batch queues starve live interactive calls.	SLO-Aware Serving: Endpoint Picker (EPP) tracks queues/KV cache to prioritize live traffic within TTFT thresholds.

Persona Tension

SRE Lead: "How do we keep competing workloads like live chat and background batch processing from disrupting one another on shared GPUs?"

Platform Engineer: "We deploy SLO-Aware Serving in the llm-d inference engine to dynamically prioritize interactive traffic over background queues"

Think First: Question

Before examining the mechanics below: How does admission control guarantee latency targets for interactive requests without starving asynchronous batch queues?

The Core Answer

"The Endpoint Picker tracks request queues and KV cache utilization to route high-priority traffic ahead of low-priority workloads."

What is it?

SLO-Aware Serving is an admission control and traffic shaping mechanism in the llm-d distributed inference engine that guarantees high-priority, latency-sensitive requests meet their Service Level Objectives (SLOs) when sharing GPUs with lower-priority workloads.

How it works?

The system introduces admission control, fairness policies, and starvation protection. The Endpoint Picker (EPP) tracks request queues and KV cache utilization. High-priority interactive requests are routed ahead of batch or low-priority workloads. It prevents any single tenant from monopolizing shared GPU resources, ensuring latency-sensitive requests (e.g., live chatbot responses) are served within their target Time-to-First-Token (TTFT) threshold while gracefully delaying asynchronous batch queues without starving them.

Enterprise Use Case Scenario

A telecommunications provider (such as Telenor) runs a shared GPU cluster serving both an interactive customer chat system (demanding low latency) and a nightly batch job summarizing customer service transcripts. During peak hours, interactive customer traffic surges. SLO-aware serving dynamically prioritizes the live chat requests over the batch processing, maintaining the SLA of the consumer chat application without needing to over-provision dedicated GPU pools.

Controlled Deployments (Canary and Rolling Updates) [GA]

Persona Tension

SRE Lead: "How do we prevent production downtime or performance degradation when transitioning between model versions?"

Platform Engineer: "Controlled Deployments allow us to safely test updates with live canary traffic and trigger automated rollbacks if performance metrics degrade."

Think First: Question

Before examining the mechanics below: What mechanism protects the end-user experience when a latency anomaly occurs during a model upgrade?

The Core Answer

"The deployment engine monitors live performance metrics and automatically triggers a rollback to the previous stable engine upon detecting anomalies."

What is it?

Controlled Deployments is a model lifecycle management feature that allows platform teams to safely deploy, test, and transition between model versions in production with zero downtime and automated rollback.

How it works?

The platform supports rolling updates, canary releases, and inference-metric-based rollback. When upgrading a model, administrators can route a small fraction (e.g., 5%) of live production traffic to the new version. The system monitors live performance metrics (request success, latency, error rates). If performance degrades below a specified threshold, the system automatically triggers an automated rollback, reverting traffic to the previous version. In-flight requests on the newer model are allowed to complete gracefully before the pods are decommissioned.

Enterprise Use Case Scenario

An e-commerce retailer is upgrading its product recommendation LLM from version 1.0 to 1.1. During the transition, a sudden spike in latency occurs on the new version due to an un-optimized serving parameter. RHAI's controlled deployment engine detects the latency anomaly and automatically rolls back all traffic to the stable v1.0 engine, protecting the customer checkout experience from disruption.

Tool Calling Through Disaggregated Serving Path [GA]

Architectural Dimension	Traditional Serving	RHAI 3.5 Innovation
Execution Topology	Combined prefill and decode execution; scaling breaks stateful connections.	Disaggregated Serving: Prefill and decode split onto dedicated pools while preserving tool-use structures.

Think First: Question

Before examining the mechanics below: Why is disaggregated serving economically advantageous for agents processing massive prompts with brief outputs?

The Core Answer

"By deploying on a disaggregated prefill/decode architecture, the firm scales prefill and decode nodes independently, maintaining real-time tool execution speeds while reducing overall GPU infrastructure costs by 40%."

What is it?

Disaggregated serving splits model execution into separate prefill (prompt processing) and decode (token generation) phases on dedicated GPU pools. In RHAI 3.5, Tool Calling is fully supported through this disaggregated architecture, so teams no longer have to choose between infrastructure scale and agent capability.

How it works?

Upstream prefill nodes ingest the initial prompt and tool-use schemas, while downstream decode nodes generate the tool call tokens. Previously, distributed cluster routing broke the stateful connection required for schema-compliant tool calls. RHAI 3.5 ensures that the disaggregated serving path correctly preserves and completes tool-use structures across node boundaries, allowing developers to scale their model-serving layer across multiple compute nodes without

breaking agent capability.

Enterprise Use Case Scenario

An insurance firm deploys a complex claim processing agent that must query multiple SQL databases and process long claim PDFs. The prompt sizes are massive (requiring high prefill compute), but the final tool call is concise (low decode compute). By deploying on a disaggregated prefill/decode architecture, the firm scales prefill and decode nodes independently, maintaining real-time tool execution speeds while reducing overall GPU infrastructure costs by 40%.

KV Cache CPU and NVMe Offloading [GA / DP]

Architectural Dimension	Traditional Memory Limits	RHAI 3.5 Innovation
Context Memory	GPU VRAM limits force context truncation or cause Out-of-Memory crashes.	Cache Offloading: Dynamic shift of KV cache to CPU DRAM (GA) and NVMe storage (DP).

Persona Tension

SRE Lead: "How do we prevent Out-of-Memory (OOM) crashes when models analyze massive documents or long multi-turn conversations?"

Platform Engineer: "KV Cache Offloading dynamically shifts cache segments from GPU VRAM to system CPU DRAM and NVMe storage, removing the memory ceiling."

Think First: Question

Before examining the mechanics below: How does offloading KV cache eliminate the need to truncate context in long-document processing?

The Core Answer

"The engine swaps inactive context blocks to CPU or NVMe storage, retrieving them back into VRAM seamlessly when requested by the user."

What is it?

KV Cache Offloading is a memory optimization capability in the vLLM engine that allows the Key-Value (KV) cache to be shifted from high-cost GPU VRAM to host system memory (CPU DRAM) and fast storage (NVMe), eliminating the "context wall" that limits conversation length.

How it works?

In RHAI 3.5, KV Cache CPU offloading is Generally Available (GA), while NVMe storage offload is in Developer Preview (DP). When an active conversation or long document ingestion exceeds the physical limits of GPU VRAM, the engine dynamically transfers older or inactive segments of the KV cache to CPU memory, or further down to a POSIX NVMe filesystem tier. When the user returns to those context blocks, the system swaps them back into VRAM. This process removes the memory ceiling that typically forces agents to truncate or summarize long conversations.

Enterprise Use Case Scenario

A legal tech corporation runs an AI document-review assistant that analyzes 500-page contract files. Normally, loading the entire context of several documents simultaneously would cause the model's pod to crash with an Out-of-Memory (OOM) error or force context truncation (degrading review accuracy). With CPU and NVMe cache offloading, the assistant retains the entire context of the legal document across multi-turn queries, ensuring high-fidelity reasoning over the complete file.

The Foundation for AI Agents that Work Beyond Text

Enterprise AI use cases are evolving to require multimodal models that process images, audio, and documents alongside text. Servicing these needs on separate infrastructure silos adds operational complexity you don't need.

Unified Multimodal Inference (vLLM Omni) [DP]

Think First: Question

Before examining the mechanics below: What is the primary operational benefit of serving vision, text, and audio models under one container configuration?

The Core Answer

"It reduces configuration overhead and eliminates cross-network latency between single-modality models."

What is it?

vLLM Omni (Developer Preview) is an extension to the RHAI model serving layer that serves multiple input/output modalities—specifically text, audio, and images—within a single, unified serving configuration.

How it works?

Instead of routing multimodal requests across separate model pipelines and infrastructure configurations, vLLM Omni hosts models capable of processing multiple modalities (e.g., vision-language models like Granite Vision) under a single KServe deployment. A request containing an image and a text prompt is received, processed, and responded to through the same container configuration, reusing the same physical GPU allocation.

Enterprise Use Case Scenario

An automotive manufacturer deploys an AI technician assistant on the factory floor. The assistant receives high-resolution photos of engine parts and voice clips describing mechanical symptoms, and responds with a text-based repair guide. With vLLM Omni, the manufacturer runs the entire application through a single, unified serving infrastructure on RHAI, reducing configuration overhead and eliminating cross-network latency between single-modality models.

Connecting Data to Models

Raw foundation models are functionally limited without access to enterprise data. RHAI 3.5 closes this gap with production-grade data integration pipelines, advanced model-tuning frameworks, and robust evaluation systems to build trustworthy AI applications.

This section focuses on:

- AutoRAG and AutoML: From Enterprise Data to Production AI
- Post-Training and Inference-Time Scaling: Improve Model Reasoning
- AI Evaluations and AI Safety: Build Confidence Before Production

AutoRAG and AutoML: From Enterprise Data to Production AI

==

Architectural Dimension		RHAI 3.5 Innovation

The Governance Rule

Persona Tension

SRE Lead:

Platform Engineer:

Think First: Question

The Core Answer

What is it?

How it works?

Enterprise Use Case Scenario

Post-Training and Inference-Time Scaling: Improve Model Reasoning

==

Architectural Dimension		RHAI 3.5 Innovation

The Governance Rule

Persona Tension

SRE Lead:

Platform Engineer:

Think First: Question

The Core Answer

What is it?

How it works?

Enterprise Use Case Scenario

AI Evaluations and AI Safety: Build Confidence Before Production

==

Architectural Dimension		RHAI 3.5 Innovation

The Governance Rule

Persona Tension

SRE Lead:

Platform Engineer:

Think First: Question

The Core Answer

What is it?

How it works?

Enterprise Use Case Scenario

Practical Agentic AI

Red Hat AI 3.5 moves beyond basic model serving—delivering a governed, secure agentic platform built for production.

This section focuses on:

- Agentic API Foundation: OGX (GA)
- Pre-built Templates in AI Hub for Developer Productivity

Agentic API Foundation: OGX (GA)

Pre-built Templates in AI Hub for Developer Productivity

Enterprise Platform Infrastructure, Scale & Security

Scaling AI operations across your organization takes solid platform governance. Red Hat OpenShift AI 3.5 unifies safety, observability, multi-tenancy, and hardware acceleration into a single operational view.

This section focuses on:

- AI Security: Automated Red Teaming and Security Insights
- Measurable Enterprise AI: Zero-Configuration Observability
- Multi-Tenancy for Enterprise AI
- GPU-as-a-Service (GPUaaS)

AI Security: Automated Red Teaming and Security Insights

Measurable Enterprise AI: Zero-Configuration Observability

Multi-Tenancy for Enterprise AI

GPU-as-a-Service (GPUaaS)

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)